main.c

```c
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    double fun(double,double);
    double b = fun(3.5,6.5);
    printf("%lf",b);
    return 0;
}
double fun(double a, double b)
{
    return a+b;
}

```

main.c

```c
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    double fun(double,double);
    double b = fun(3.5,6.5);
    printf("%lf",b);
    return 0;
}
double fun(double a, double b)
{
    return a+b;
}

```

input

```
Hello World
10.000000

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c

```c
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    int b = fun(3.5,6.5);
    printf("%d",b);
    return 0;
}
int fun(double a, double b)
{
    int c = a+b;
    return c;
}

```

- In old C (C89) → The compiler would assume fun returns int by default (an implicit declaration) and try to match the arguments loosely. This causes undefined behavior and warnings.
- In modern C (C99 and later, including C11, C17, C23) → implicit function declaration is not allowed. This means the compiler will give a compilation error.

input

```
main.c: In function 'main':
main.c:5:13: warning: implicit declaration of function 'fun' [-Wimplicit-function-declaration]
    5 |     int b = fun(3.5,6.5);
      |             ^~~
Hello World
10

...Program finished with exit code 0
Press ENTER to exit console.
```

main.c

```c
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    int b = fun(3.5,6.5);
    printf("%d",b);
    return 0;
}
double fun(double a, double b)
{
    return a+b;
}
```

input

Compilation failed due to following error(s).

```
main.c: In function ‘main’:
main.c:5:13: warning: implicit declaration of function ‘fun’ [-Wimplicit-function-declaration]
    5 |     int b = fun(3.5,6.5);
      |             ^~~
main.c: At top level:
main.c:9:8: error: conflicting types for ‘fun’; have ‘double(double,  double)’
    9 | double fun(double a, double b)
      |        ^~~
main.c:5:13: note: previous implicit declaration of ‘fun’ with type ‘int()’
    5 |     int b = fun(3.5,6.5);
      |             ^~~
```

C Vs C++ - function prototype

- Both in **C** and **C++**, if function is called only after function definition, then it is not necessary to have function prototype
  - Function definition works as function prototype
- In **C++**, function definition or prototype is must before function call
  - Function **name, return type and arguments** (number, order and type) in prototype must be same as function definition.
- If function prototype has empty parentheses
  - in C, it means any number of arguments
  - while in C++ it means **void** as argument (required for function overloading)

C Vs C++ - function prototype

- In C, function definition or prototype is not must before function call
  - In absence of function definition and prototype function call works as implicit declaration (**with warning**)
    - It assumes return type to be **int**
    - And number, order and types of arguments is derived from values passed for function call
    - If definition and implicit declaration mismatch, then it results in **error**.

* this was support only in old C (C89) not In modern C (C99 and later, including C11, C17, C23)

main.c

```c
#include <stdio.h>
int main()
{
        void func();
        func(3,4);
        return 0;
}
void func(int a, int b)
{
        printf("Nothing\n");
}

```

input

```
Nothing


...Program finished with exit code 0
Press ENTER to exit console.
```

main.cpp

```cpp
#include <iostream>
using namespace std;
int main()
{
        void func();
        func(3,4);
        return 0;
}
void func(int a, int b)
{
        cout<< "Nothing";
}
```

input

Compilation failed due to following error(s).

```
main.cpp: In function ‘int main()’:
main.cpp:6:13: error: too many arguments to function ‘void func()’
    6 |         func(3,4);
      |         ~~~~^~~~~
main.cpp:5:14: note: declared here
    5 |         void func();
      |              ^~~~
```

C++ Inline functions

- Function like macros (C also supports this)

```
#include<iostream>

#define SQUARE(x) x * x

using namespace std;

int sqr(int x)
{
	return x * x;
}
int main()
{
	cout << SQUARE(3) << endl;
	cout << SQUARE(3.1 + 1) << endl << endl;

	cout << sqr(3.1) << endl;
	cout << sqr(3 + 1) << endl;

	return 0;
}
```

C++ Inline functions

- Benefit of Macros (~Drawback of function)
  - No function call needed, hence no overhead (like required for function call)

- Drawbacks of Macros (~Benefits of function)
  - Sometimes you get result which you might not have expected
  - No type checking

C++ Inline functions

- Concept of **inline** functions have been introduced to benefit from better of both worlds – macros and functions

```
#include<iostream>
using namespace std;
inline int cube(int n)
{
	return n * n * n;
}

int main()
{
	float ans1 = cube(3);
	float ans2 = cube(3.1 + 1);
	cout << ans1 << endl;
	cout << ans2 << endl;
	return 0;
}
```

C++ Inline functions

- When a function is defined as **inline**, it is a just a **request (NO GUARANTEE)** to the compiler to expand it in line when it is called.
  - It is **as if** whole code of the inline function gets inserted or substituted at the point of inline function call.
  - Compiler is intelligent - Same variable names in calling function and inline function will not clash
  - Making function inline will not change the final output at all (Except performance difference due to inlining)
  - Inline function must be defined before it is called – (so that compiler knows exact code)

- Inline functions are not same as macros – macros are handled by preprocessor and inline functions are handled by compiler

C++ Inline functions

- Only small function (one/two lines) should be defined as inline
- There is a trade-off – performance Vs size of output file (binary file)
  - If you inline function (specifically larger functions) at many locations, then size of output binary file will increase as code will be repeated at multiple locations

- inline function **CAN NOT**
  - be recursive in nature
  - contain static variable
  - contain loop, switch or goto

C++ functions - default arguments

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3 = 111, int n4 = 222);
int main()
{
        cout << sum(1, 2, 3, 4) << endl;     // 10
        cout << sum(1, 2, 3) << endl;    // 228
        cout << sum(1, 2) << endl;        // 336
        return 0; }
int sum(int n1, int n2, int n3, int n4)
{
        return n1 + n2 + n3 + n4;
}
```

C++ functions - default arguments

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3 = 111, int n4 = 222);
int main()
{
        cout << sum(1, 2, 3, 4) << endl;     // 10
        cout << sum(1, 2, 3) << endl;   // 228
        cout << sum(1, 2) << endl;       // 336
        return 0; }
int sum(int n1, int n2, int n3, int n4)
{
        return n1 + n2 + n3 + n4;
}
```

- It is also valid to provide default values for all arguments

  int sum(int n1 = 11, int n2 = 33, int n3 = 44, int n4 = 66);

- int sum(int n1, int n2, int n3 = 4, int n4); // **Invalid**

- You can assign **default values** to function parameters in the **prototype (declaration)**.

```
int sum(int n1, int n2, int n3 = 111, int n4 = 222);
```

Here, if you don't provide **n3** or **n4**, they'll automatically take **111** and **222**.

- The **definition** of the function should **not repeat the defaults** (they go in declaration only).
- Default arguments must be assigned **from right to left** (you can't skip middle ones).

```
int sum(int n1, int n2, int n3 = 4, int n4);
```

- This is invalid because **n3** has a default but **n4** does not.
- **Rule:** once you start giving defaults, all parameters to the right must also have defaults.

C++ functions - default arguments

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3 = 3, int n4 = 4);
void fun()
{
	cout << "fun" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
}
int main()
{
	int sum(int n1, int n2, int n3 = 111, int n4 = 222);
	cout << "main" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
	fun();
	return 0;
}
int sum(int n1, int n2, int n3 = 3, int n4 = 4)
{
	return n1 + n2 + n3 + n4;
}
```

C++ functions - default arguments

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3 = 3, int n4 = 4);
void fun()
{
	cout << "fun" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
}
int main()
{
	int sum(int n1, int n2, int n3 = 111, int n4 = 222);
	cout << "main" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
	fun();
	return 0;
}
int sum(int n1, int n2, int n3 = 3, int n4 = 4) // Error – You can not assign default values in definition and declaration
{                                               // if definition and declaration are in the same socpe
	return n1 + n2 + n3 + n4;
}
```

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3, int n4);
void fun()
{
	cout << "fun" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
}
int main()
{
	int sum(int n1, int n2, int n3 = 111, int n4 = 222);
	cout << "main" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
	fun();
	return 0;
}
int sum(int n1, int n2, int n3 = 3, int n4 = 4)
{
	return n1 + n2 + n3 + n4;
}
```

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3, int n4);
void fun()
{
	cout << "fun" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;   // Error – No default values until after definition of sum function
	cout << sum(1, 2) << endl;          // Error – No default values until after definition of sum function
}
int main()
{
	int sum(int n1, int n2, int n3 = 111, int n4 = 222);
	cout << "main" << endl;
	cout << sum(1, 2, 3, 4) << endl;
	cout << sum(1, 2, 3) << endl;
	cout << sum(1, 2) << endl;
	fun();
	return 0;
}
int sum(int n1, int n2, int n3 = 3, int n4 = 4)
{
	return n1 + n2 + n3 + n4;
}
```

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3, int n4);
int main()
{
        int sum(int n1, int n2, int n3 = 111, int n4 = 222);
        cout << "main" << endl;
        cout << sum(1, 2, 3, 4) << endl;
        cout << sum(1, 2, 3) << endl;          // Will get default values from local declaration
        cout << sum(1, 2) << endl;             // Will get default values from local declaration
        void fun(); fun();
        return 0;
}
int sum(int n1, int n2, int n3 = 3, int n4 = 4)
{
        return n1 + n2 + n3 + n4;
}
void fun()
{
        cout << "fun" << endl;
        cout << sum(1, 2, 3, 4) << endl;
        cout << sum(1, 2, 3) << endl;          // Default values from definition
        cout << sum(1, 2) << endl;             // Default values from definition
}
```

```
main
10
228
336
fun
10
10
10
```

```
#include<iostream>
using namespace std;
int sum(int n1, int n2, int n3 = 3, int n4 = 4);
int main()
{
        int sum(int n1, int n2, int n3 = 111, int n4 = 222);
        cout << "main" << endl;
        cout << sum(1, 2, 3, 4) << endl;
       cout << sum(1, 2, 3) << endl;       // Will get default values from local declaration
       cout << sum(1, 2) << endl;          // Will get default values from local declaration
        void fun(); fun();
        return 0;
}
int sum(int n1, int n2, int n3, int n4)
{
        return n1 + n2 + n3 + n4;
}
void fun()
{
        cout << "fun" << endl;
        cout << sum(1, 2, 3, 4) << endl;
        cout << sum(1, 2, 3) << endl;       // Default values from global declaration of sum function still hold
        cout << sum(1, 2) << endl;             // Even after definition of sum function without default values
}
```

```
main
10
228
336
fun
10
10
10
```

C++ functions - default arguments

- If function prototype and function definition are in global scope
  - Default values be provided with either definition or prototype, not both (even if you provide same values)
    - If default values are in definition then until function definition, function would be treated as if it does not have default values (as prototype does not have default values)
    - If default values are in prototype, then default values from prototype would be applicable through out (even after definition, even though definition does not have default values)
- If prototype is within another function and definition in global scope
  - It is allowed to have default values in both definition and prototype
    - Default values from locally declared prototype will have priority over default values from definition in global scope

- **Avoid default values in definition, have default values in prototype**